Introduction

API endpoint

```
https://api.monzo.com
```

Examples in this documentation are written using httpie for clarity.

To install `httpie` on macOS run `brew install httpie`

The Monzo API is designed to be a predictable and intuitive interface for interacting with users' accounts. We offer both a REST API and webhooks.

The Developers category on our forum is the place to get help with our API, discuss ideas, and show off what you build.

The Monzo Developer API is not suitable for building public applications.
You may only connect to your own account or those of a small set of users you explicitly allow.

Looking for our Open Banking API documentation?
This has now moved to a new page. For firms authorised as Account Information Service Providers under PSD2 we offer an AISP API. We also offer an API for authorised PISPs and CBPIIs.

Authentication

The Monzo API implements OAuth 2.0 to allow users to log in to applications without exposing their credentials. The process involves several steps:

1. **Acquire** an access token, and optionally a refresh token
2. **Use** the access token to make authenticated requests
3. If you were issued a refresh token: **refresh** the access token when it expires

Strong Customer Authentication
We don't grant any permissions to the access token until the owner of the account that your client wants to access has approved access to their data in the Monzo app. Your user will receive a push notification after authenticating with their email.

To get started quickly, you can use the access token from the API playground and avoid implementing the OAuth login flow.

Before you begin, you will need to create a client in the developer tools.

CLIENT CONFIDENTIALITY

Clients are designated either confidential or non-confidential.

- **Confidential** clients keep their client secret hidden. For example, a server-side app that never exposes its secret to users.
- **Non-confidential** clients cannot keep their client secret hidden. For example, client-side apps that store their client secret on the user's device, where it could be intercepted.

  Non-confidential clients are not issued refresh tokens.

Acquire an access token

Acquiring an access token is a three-step process:

1. Redirect the user to Monzo to authorise your app
2. Monzo redirects the user back to your app with an authorization code
3. Exchange the authorization code for an access token.

This access token doesn't have any permissions until your user has approved access to their data in the Monzo app.

REDIRECT THE USER TO MONZO

```
"https://auth.monzo.com/?
   client_id=$client_id&
   redirect_uri=$redirect_uri&
   response_type=code&
   state=$state_token"
```

Send the user to Monzo in a web browser, where they will log in and grant access to their account.

Strong Customer Authentication
In addition to authenticating with email, the user will receive a notification in their Monzo app to ask them to authenticate with their card PIN, fingerprint or Face ID.

URL ARGUMENTS

Parameter	Description
`client_id` Required	Your client ID.
`redirect_uri` Required	A URI to which users will be redirected after authorising your app.
`response_type` Required	Must be set to `code`.
`state`	An unguessable random string used to protect against cross-site request forgery attacks.

MONZO REDIRECTS BACK TO YOUR APP

```
"https://your.example.com/oauth/callback?
    code=$authorization_code&
    state=$state_token"
```

If the user allows access to their account, Monzo redirects them back to your app.

URL ARGUMENTS

Parameter	Description
`code`	A temporary authorization code which will be exchanged for an access token in the next step.
`state`	The same string you provided as `state` when sending the user to Monzo. If this value differs from what you sent, you **must** abort the authentication process.

EXCHANGE THE AUTHORIZATION CODE

```
$ http --form POST "https://api.monzo.com/oauth2/token" \
    "grant_type=authorization_code" \
    "client_id=$client_id" \
    "client_secret=$client_secret" \
    "redirect_uri=$redirect_uri" \
    "code=$authorization_code"
```

```
{
    "access_token": "access_token",
    "client_id": "client_id",
    "expires_in": 21600,
    "refresh_token": "refresh_token",
    "token_type": "Bearer",
    "user_id": "user_id"
}
```

When you receive an authorization code, exchange it for an access token. The resulting access token is tied to both your client and an individual Monzo user, and is valid for several hours.

REQUEST ARGUMENTS

Parameter	Description
`grant_type` Required	This must be set to `authorization_code`
`client_id` Required	The client ID you received from Monzo.
`client_secret` Required	The client secret which you received from Monzo.
`redirect_uri` Required	The URL in your app where users were sent after authorisation.
`code` Required	The authorization code you received when the user was redirected back to your app.

Authenticating requests

```
$ http "https://api.monzo.com/ping/whoami" \
    "Authorization: Bearer $access_token"
```

```
{
    "authenticated": true,
    "client_id": "client_id",
    "user_id": "user_id"
}
```

All requests must be authenticated with an access token supplied in the `Authorization` header using the `Bearer` scheme. Your client may only have *one* active access token at a time, per user. Acquiring a new access token will invalidate any other token you own for that user.

To get information about an access token, you can call the `/ping/whoami` endpoint.

Refreshing access

```
$ http --form POST "https://api.monzo.com/oauth2/token" \
    "grant_type=refresh_token" \
    "client_id=$client_id" \
    "client_secret=$client_secret" \
    "refresh_token=$refresh_token"
```

```
{
    "access_token": "access_token_2",
    "client_id": "client_id",
    "expires_in": 21600,
    "refresh_token": "refresh_token_2",
    "token_type": "Bearer",
    "user_id": "user_id"
}
```

To limit the window of opportunity for attackers in the event an access token is compromised, access tokens expire after a number of hours. To gain long-lived access to a user's account, it's necessary to "refresh" your access when it expires using a refresh token. Only "confidential" clients are issued refresh tokens – "public" clients must ask the user to re-authenticate.

Refreshing an access token will invalidate the previous token, if it is still valid. Refreshing is a one-time operation.

REQUEST ARGUMENTS

Parameter	Description
`grant_type` Required	Should be `refresh_token`.
`client_id` Required	Your client ID.
`client_secret` Required	Your client secret.
`refresh_token` Required	The refresh token received along with the original access token.

Log Out

```
$ http --form POST "https://api.monzo.com/oauth2/logout" \
    "Authorization: Bearer $access_token"
```

While access tokens do expire after a number of hours, you may wish to invalidate the token instantly at a specific time such as when a user chooses to log out of your application.

Once invalidated, the user must go through the authentication process again. You will not be able to refresh the access token.

Pagination

Endpoints which enumerate objects support time-based and cursor-based pagination.

REQUEST ARGUMENTS

Parameter	Description
`limit` Optional	Limits the number of results per-page. Default: 30 Maximum: 100.
`since` Optional	An RFC 3339-encoded timestamp. eg. `2009-11-10T23:00:00Z` …or an object id. eg. `tx_00008zhJ3kE6c8kmsGUKgn`
`before` Optional	An RFC 3339 encoded-timestamp `2009-11-10T23:00:00Z`

Expanding objects

Some objects contain the id of another object in their response. To save a round-trip, some of these objects can be expanded inline with the `expand[]` argument, which is repeatable. Objects that can be expanded are noted in individual endpoint documentation.

Accounts

Accounts represent a store of funds, and have a list of transactions.

List accounts

Returns a list of accounts owned by the currently authorised user.

```
$ http "https://api.monzo.com/accounts" \
    "Authorization: Bearer $access_token"
```

```
{
    "accounts": [
        {
            "id": "acc_00009237aqC8c5umZmrRdh",
            "description": "Peter Pan's Account",
            "created": "2015-11-13T12:17:42Z"
        }
    ]
}
```

To filter by either prepaid or current account, add `account_type` as a url parameter. Valid `account_type`s are `uk_retail`, `uk_retail_joint`.

```
$ http "https://api.monzo.com/accounts" \
    "Authorization: Bearer $access_token" \
    account_type==uk_retail
```

Balance

Retrieve information about an account's balance.

Read balance

```
$ http "https://api.monzo.com/balance" \
    "Authorization: Bearer $access_token" \
    "account_id==$account_id"
```

```
{
    "balance": 5000,
    "total_balance": 6000,
    "currency": "GBP",
    "spend_today": 0
}
```

Returns balance information for a specific account.

REQUEST ARGUMENTS

Parameter	Description
`account_id` Required	The id of the account.

RESPONSE ARGUMENTS

Parameter	Description
`balance`	The currently available balance of the account, as a 64bit integer in minor units of the currency, eg. pennies for GBP, or cents for EUR and USD.
`total_balance`	The sum of the currently available balance of the account and the combined total of all the user's pots.
`currency`	The ISO 4217 currency code.
`spend_today`	The amount spent from this account today (considered from approx 4am onwards), as a 64bit integer in minor units of the currency.

Pots

A pot is a place to keep some money separate from the main spending account.

List pots

```
$ http "https://api.monzo.com/pots" \
    "current_account_id==$account_id" \
    "Authorization: Bearer $access_token"
```

```
{
  "pots": [
    {
      "id": "pot_0000778xxfgh4iu8z83nWb",
      "name": "Savings",
      "style": "beach_ball",
      "balance": 133700,
      "currency": "GBP",
      "created": "2017-11-09T12:30:53.695Z",
      "updated": "2017-11-09T12:30:53.695Z",
      "deleted": false
    }
  ]
}
```


Returns a list of pots owned by the currently authorised user that are associated with the specified account.

Deposit into a pot

```
$ http --form PUT "https://api.monzo.com/pots/$pot_id/deposit" \
    "Authorization: Bearer $access_token" \
    "source_account_id=$account_id" \
    "amount=$amount" \
    "dedupe_id=$dedupe_id"
```

Move money from an account owned by the currently authorised user into one of their pots.


```
{
    "id": "pot_00009exampleP0tOxWb",
    "name": "Wedding Fund",
    "style": "beach_ball",
    "balance": 550100,
    "currency": "GBP",
    "created": "2017-11-09T12:30:53.695Z",
    "updated": "2018-02-26T07:12:04.925Z",
    "deleted": false
}
```


REQUEST ARGUMENTS

Parameter	Description
`source_account_id` Required	The id of the account to withdraw from.
`amount` Required	The amount to deposit, as a 64bit integer in minor units of the currency, eg. pennies for GBP, or cents for EUR and USD.
`dedupe_id` Required	A unique string used to de-duplicate deposits. Ensure this remains static between retries to ensure only one deposit is created.

RESPONSE ARGUMENTS

Parameter	Description
`id`	The pot id.
`name`	The pot name.
`style`	The pot background image.
`balance`	The new pot balance.
`currency`	The pot currency.
`created`	When this pot was created.
`updated`	When this pot was last updated.
`deleted`	Whether this pot is deleted. The API will be updated soon to not return deleted pots.

Withdraw from a pot

```
$ http --form PUT "https://api.monzo.com/pots/$pot_id/withdraw" \
    "Authorization: Bearer $access_token" \
    "destination_account_id=$account_id" \
    "amount=$amount" \
    "dedupe_id=$dedupe_id"
```

Move money from a pot owned by the currently authorised user into one of their accounts.


```
{
    "id": "pot_00009exampleP0tOxWb",
    "name": "Flying Lessons",
    "style": "blue",
    "balance": 350000,
```

```
    "currency": "GBP",
    "created": "2017-11-09T12:30:53.695Z",
    "updated": "2018-02-26T07:12:04.925Z",
    "deleted": false
}
```

Added Security
If the pot is protected with added security, withdrawals cannot be made via this API and must be made in-app.

REQUEST ARGUMENTS

Parameter	Description
`destination_account_id` Required	The id of the account to deposit into.
`amount` Required	The amount to deposit, as a 64bit integer in minor units of the currency, eg. pennies for GBP, or cents for EUR and USD.
`dedupe_id` Required	A unique string used to de-duplicate deposits. Ensure this remains static between retries to ensure only one withdrawal is created.

RESPONSE ARGUMENTS

Parameter	Description
`id`	The pot id.
`name`	The pot name.
`style`	The pot background image.
`balance`	The new pot balance.
`currency`	The pot currency.
`created`	When this pot was created.
`updated`	When this pot was last updated.
`deleted`	Whether this pot is deleted. The API will be updated soon to not return deleted pots.

Transactions

Transactions are movements of funds into or out of an account. Negative transactions represent debits (ie. *spending* money) and positive transactions represent credits (ie. *receiving* money).

Most properties on transactions are self-explanatory. We'll eventually get around to documenting them all, but in the meantime let's discuss the most interesting/confusing ones:

PROPERTIES

Property	Description
`amount`	The amount of the transaction in minor units of `currency`. For example pennies in the case of GBP. A negative amount indicates a debit (most card transactions will have a negative amount)
`decline_reason`	**This is only present on declined transactions!** Valid values are `INSUFFICIENT_FUNDS`, `CARD_INACTIVE`, `CARD_BLOCKED`, `INVALID_CVC` or `OTHER`.
`is_load`	Top-ups to an account are represented as transactions with a positive amount and `is_load = true`. Other transactions such as refunds, reversals or chargebacks may have a positive amount but `is_load = false`
`settled`	The timestamp at which the transaction settled. In most cases, this happens 24-48 hours after `created`. If this field is an empty string, the transaction is authorised but not yet "complete."
`category`	The category can be set for each transaction by the user. Over time we learn which merchant goes in which category and auto-assign the category of a transaction. If the user hasn't set a category, we'll return the default category of the merchant on this transactions. Top-ups have category `mondo`. Valid values are `general`, `eating_out`, `expenses`, `transport`, `cash`, `bills`, `entertainment`, `shopping`, `holidays`, `groceries`.
`merchant`	This contains the `merchant_id` of the merchant that this transaction was made at. If you pass `?expand[]=merchant` in your request URL, it will contain lots of information about the merchant.

Retrieve transaction

```
$ http "https://api.monzo.com/transactions/$transaction_id" \
    "Authorization: Bearer $access_token" \
    # Here we are expanding the merchant \
    "expand[]==merchant"
```

```
{
    "transaction": {
        "amount": -510,
        "created": "2015-08-22T12:20:18Z",
```

```
        "currency": "GBP",
        "description": "THE DE BEAUVOIR DELI C LONDON       GBR",
        "id": "tx_00008zIcpb1TB4yeIFXMzx",
        "merchant": {
            "address": {
                "address": "98 Southgate Road",
                "city": "London",
                "country": "GB",
                "latitude": 51.54151,
                "longitude": -0.08482400000002599,
                "postcode": "N1 3JD",
                "region": "Greater London"
            },
            "created": "2015-08-22T12:20:18Z",
            "group_id": "grp_00008zIcpbBOaAr7TTP3sv",
            "id": "merch_00008zIcpbAKe8shBxXUtl",
            "logo": "https://pbs.twimg.com/profile_images/527043602623389696/68_SgUWJ.jpeg",
            "emoji": "🍞",
            "name": "The De Beauvoir Deli Co.",
            "category": "eating_out"
        },
        "metadata": {},
        "notes": "Salmon sandwich 🍞",
        "is_load": false,
        "settled": "2015-08-23T12:20:18Z"
    }
}
```


Returns an individual transaction, fetched by its id.

REQUEST ARGUMENTS

Parameter	Description
`expand[]` Repeated	Can be `merchant`.

List transactions

```
$ http "https://api.monzo.com/transactions" \
    "Authorization: Bearer $access_token" \
    "account_id==$account_id"
```


```
{
    "transactions": [
        {
            "amount": -510,
            "created": "2015-08-22T12:20:18Z",
            "currency": "GBP",
            "description": "THE DE BEAUVOIR DELI C LONDON       GBR",
            "id": "tx_00008zIcpb1TB4yeIFXMzx",
            "merchant": "merch_00008zIcpbAKe8shBxXUtl",
            "metadata": {},
            "notes": "Salmon sandwich 🍞",
            "is_load": false,
            "settled": "2015-08-23T12:20:18Z",
            "category": "eating_out"
        },
        {
            "amount": -679,
            "created": "2015-08-23T16:15:03Z",
            "currency": "GBP",
            "description": "VUE BSL LTD           ISLINGTON     GBR",
            "id": "tx_00008zL2INM3xZ41THuRF3",
            "merchant": "merch_00008z6uFVhVBcaZzSQwCX",
            "metadata": {},
            "notes": "",
            "is_load": false,
            "settled": "2015-08-24T16:15:03Z",
            "category": "eating_out"
        },
    ]
}
```


Returns a list of transactions on the user's account.

> **Strong Customer Authentication**
> After a user has authenticated, your client can fetch all of their transactions, and after 5 minutes, it can only sync the last 90 days of transactions. If you need the user's entire transaction history, you should consider fetching and storing it right after authentication.

REQUEST ARGUMENTS

Parameter	Description
`account_id` Required	The account to retrieve transactions from.
`since` Optional	Start time as RFC3339 encoded timestamp (`2009-11-10T23:00:00Z`)
`before` Optional	End time time as RFC3339 encoded timestamp (`2009-11-10T23:00:00Z`)

Parameter	Description
Pagination Optional	This endpoint can be paginated.

Annotate transaction

```
$ http --form PATCH "https://api.monzo.com/transactions/$transaction_id" \
    "Authorization: Bearer $access_token" \
    "metadata[$key1]=$value1" \
    # Set a key's value as empty to delete it
    "metadata[$key2]="
```


```
{
    "transaction": {
        "amount": -679,
        "created": "2015-08-23T16:15:03Z",
        "currency": "GBP",
        "description": "VUE BSL LTD            ISLINGTON     GBR",
        "id": "tx_00008zL2INM3xZ41THuRF3",
        "merchant": "merch_00008z6uFVhVBcaZzSQwCX",
        "metadata": {
            "foo": "bar"
        },
        "notes": "",
        "is_load": false,
        "settled": "2015-08-24T16:15:03Z",
        "category": "eating_out"
    }
}
```


You may store your own key-value annotations against a transaction in its `metadata`.

REQUEST ARGUMENTS

Parameter	Description
`metadata[$name]` Repeated	Include each key you would like to modify. To delete a key, set its value to an empty string.

> Metadata is private to your application. Updating the `notes` key applies the value to the transaction's top-level notes property.

Feed items

The Monzo app is organised around the feed – a reverse-chronological stream of events. Transactions are one such feed item, and your application can create its own feed items to surface relevant information to the user.

It's important to keep a few principals in mind when creating feed items:

1. Feed items are *discrete* events that happen at a *point in time*.
2. Because of their prominence within the Monzo app, feed items should contain information of *high value*.
3. While the appearance of feed items can be customised, care should be taken to match the style of the Monzo app so that your feed items feel part of the experience.

Create feed item

```
$ http --form POST "https://api.monzo.com/feed" \
    "Authorization: Bearer $access_token" \
    "account_id=$account_id" \
    "type=basic" \
    "url=https://www.example.com/a_page_to_open_on_tap.html" \
    "params[title]=My custom item" \
    "params[image_url]=www.example.com/image.png" \
    "params[background_color]=#FCF1EE" \
    "params[body_color]=#FCF1EE" \
    "params[title_color]=#333333" \
    "params[body]=Some body text to display"
```


```
{}
```


Creates a new feed item on the user's feed. These can be dismissed.

REQUEST ARGUMENTS (FOR ALL FEED ITEM TYPES)

Parameter	Description
`account_id` Required	The account to create a feed item for.

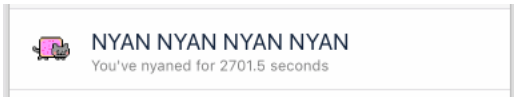
Parameter	Description
`type` Required	Type of feed item. Currently only `basic` is supported.
`params` Required	A *map* of parameters which vary based on `type`
`url` Optional	A URL to open when the feed item is tapped. If no URL is provided, the app will display a fallback view based on the title & body.

PER-TYPE ARGUMENTS

Each type of feed item supports customisation with a specific list of `params`. Currently we only support creation of the `basic` feed item which requires the parameters below. These should be sent as form parameters as in the example to the right.

BASIC

The basic type displays an `image`, with `title` text and optional `body` text.
Note the image supports animated gifs!



REQUEST ARGUMENTS

Parameter	Description
`title` Required	The title to display.
`image_url` Required	URL of the image to display. This will be displayed as an icon in the feed, and on the expanded page if no `url` has been provided.
`body` Optional	The body text of the feed item.
`background_color` Optional	Hex value for the background colour of the feed item in the format `#RRGGBB`. Defaults to to standard app colours (ie. white background).
`title_color` Optional	Hex value for the colour of the title text in the format `#RRGGBB`. Defaults to standard app colours.
`body_color` Optional	Hex value for the colour of the body text in the format `#RRGGBB`. Defaults to standard app colours.

Attachments

Images (eg. receipts) can be attached to transactions by uploading these via the `attachment` API. Once an attachment is *registered* against a transaction, the image will be shown in the transaction detail screen within the Monzo app.

There are two options for attaching images to transactions - either Monzo can host the image, or remote images can be displayed.

If Monzo is hosting the attachment the upload process consists of three steps:

1. Obtain a temporary authorised URL to upload the attachment to.
2. Upload the file to this URL.
3. Register the attachment against a `transaction`.

If you are hosting the attachment, you can simply register the attachment with the transaction:

1. Register the attachment against a `transaction`.

Upload attachment

The first step when uploading an attachment is to obtain a temporary URL to which the file can be uploaded. The response will include a `file_url` which will be the URL of the resulting file, and an `upload_url` to which the file should be uploaded to.

```
$ http --form POST "https://api.monzo.com/attachment/upload" \
    "Authorization: Bearer $access_token" \
    "file_name=foo.png" \
    "file_type=image/png" \
    "content_length=12345"
```

```
{
    "file_url":"https://s3-eu-west-1.amazonaws.com/mondo-image-uploads/user_00009237hliZellUicKuG1/LcCu4ogv1xW28OCcvOTL-foo.png",
    "upload_url":"https://mondo-image-uploads.s3.amazonaws.com/user_00009237hliZellUicKuG1/LcCu4ogv1xW28OCcvOTL-foo.png?AWSAccessKeyId={EXAMPLE_AWS_ACCESS_KEY_ID}\u0026Expires=1447353431\u0026Signature=k2QeDCCQQHaZeynzYKckejqXRGU%
}
```

REQUEST ARGUMENTS

Parameter	Description
`file_name` Required	The name of the file to be uploaded
`file_type` Required	The content type of the file
`content_length` Required	The HTTP Content-Length of the upload request body, in bytes.

RESPONSE ARGUMENTS

Parameter	Description
`file_url`	The URL of the file once it has been uploaded
`upload_url`	The URL to `POST` the file to when uploading

Register attachment

Once you have obtained a URL for an attachment, either by uploading to the `upload_url` obtained from the `upload` endpoint above or by hosting a remote image, this URL can then be registered against a transaction. Once an attachment is registered against a transaction this will be displayed on the detail page of a transaction within the Monzo app.

```
$ http --form POST "https://api.monzo.com/attachment/register" \
    "Authorization: Bearer $access_token" \
    "external_id=tx_00008zIcpb1TB4yeIFXMzx" \
    "file_type=image/png" \
    "file_url=https://s3-eu-west-1.amazonaws.com/mondo-image-uploads/user_00009237hliZellUicKuG1/LcCu4ogv1xW28OCcvOTL-foo.png"
```

```
{
    "attachment": {
        "id": "attach_00009238aOAIvVqfb9LrZh",
        "user_id": "user_00009238aMBIIrS5Rdncq9",
        "external_id": "tx_00008zIcpb1TB4yeIFXMzx",
        "file_url": "https://s3-eu-west-1.amazonaws.com/mondo-image-uploads/user_00009237hliZellUicKuG1/LcCu4ogv1xW28OCcvOTL-foo.png",
        "file_type": "image/png",
        "created": "2015-11-12T18:37:02Z"
    }
}
```

REQUEST ARGUMENTS

Parameter	Description
`external_id` Required	The id of the `transaction` to associate the `attachment` with.
`file_url` Required	The URL of the uploaded attachment.
`file_type` Required	The content type of the attachment.

RESPONSE ARGUMENTS

Parameter	Description
`id`	The ID of the attachment. This can be used to deregister at a later date.
`user_id`	The id of the `user` who owns this `attachment`.
`external_id`	The id of the `transaction` to which the `attachment` is attached.
`file_url`	The URL at which the `attachment` is available.
`file_type`	The file type of the `attachment`.
`created`	The timestamp *in UTC* when the attachment was created.

Deregister attachment

To remove an `attachment`, simply deregister this using its `id`

```
$ http --form POST "https://api.monzo.com/attachment/deregister" \
    "Authorization: Bearer $access_token" \
    "id=attach_00009238aOAIvVqfb9LrZh"
```

```
{}
```

REQUEST ARGUMENTS

Parameter	Description
`id` Required	The id of the `attachment` to deregister.

Receipts

Receipts are line-item purchase data added to a transaction. They contain all the information about the purchase, including the products you bought, any taxes that were added on, and how you paid. They can also contain extra details about the merchant you spent money at, such as how to contact them, but this may not appear in the app yet.

This is the API currently used by Flux to show receipts at selected retailers in your Monzo app.

PROPERTIES

```
{
    "transaction_id": "tx_00...",
    "external_id": "Order-12345678",
    "total": 1299,
    "currency": "GBP",
    "items": [],
    "taxes": [],
    "payments": [],
    "merchant": {}
}
```

Property	Description
`id`	A unique identifier generated by Monzo when you submit the receipt.
`external_id` Required	A unique identifier generated by you, which is used as an idempotency key. You might use an order number for example.
`transaction_id` Required	The ID of the Transaction to associate the Receipt with.
`total` Required	The amount of the transaction in minor units of `currency`. For example pennies in the case of GBP. The amount should be positive.
`currency` Required	Usually `GBP`, for Pounds Sterling.
`items` Required	A list of Items detailing the products included in the total.
`taxes`	A list of Taxes (e.g. VAT) added onto the total.
`payments`	A list of Payments, indicating how the customer paid the total.
`merchant`	The Merchant you shopped at. (This is a different type of object than the Merchant on a Transaction.)

Receipt Items

```
[
    {
        "description": "Burger",
        "quantity": 1,
        "unit": "",
        "amount": 539,
        "currency": "GBP",
        "tax": 77,
        "sub_items": [
            {
                "description": "Extra cheese",
                "quantity": 1,
                "unit": "",
                "amount": 100,
                "currency": "GBP",
                "tax": 0
            },
            {
                "description": "Free extra topping promotion",
                "quantity": 1,
                "unit": "",
                "amount": -100,
                "currency": "GBP",
                "tax": 0
            }
        ]
    },
    {
        "description": "Fries",
        "quantity": 1,
        "unit": "",
        "amount": 139,
        "currency": "GBP",
        "tax": 19
    },
    {
        "description": "Milkshake",
```

```
        "quantity": 2,
        "unit": "",
        "amount": 198,
        "currency": "GBP",
        "tax": 38
    },
    {
        "description": "Bananas, £1 per kg",
        "quantity": 0.3,
        "unit": "kg",
        "amount": 30,
        "currency": "GBP",
        "tax": 0
    }
]
```

Items detail each product that was included in the transaction. They let you see more detailed data in your Monzo feed than just how much you spent! 🎉

Items can be made up of sub-items, for example an extra topping on a burger. Sub-items have the same format as items (but they cannot in turn have their own sub-items!). The amounts of the sub-items should add up to amount on the item.

All of the items together, plus the taxes, should add up to the Receipt total.

PROPERTIES

Property	Description
`description` Required	The product you bought!
`amount` Required	The amount paid for the item, in pennies. If there are sub-items, this should be the total of their amounts.
`currency` Required	e.g. GBP
`quantity`	A number indicating how many of the product were bought, e.g. `2`. A floating-point number, so it can represent weights like `1.23` for example
`unit`	The unit the quantity is measured in, e.g. `kg`
`tax`	The tax, in pennies.
`sub_items`	A list of sub-items, as described above

Receipt Taxes

```
[
    {
        "description": "VAT",
        "amount": 10,
        "currency": "GBP",
        "tax_number": "945719291"
    }
]
```

Taxes will be shown near the bottom of the receipt, just above the total.

PROPERTIES

Property	Description
`description` Required	e.g. “VAT”
`amount` Required	Total amount of the tax, in pennies
`currency` Required	e.g. GBP
`tax_number`	e.g. “945719291”

Receipt Payments

```
[
    {
      "type": "card",
      "bin": "543210",
      "last_four": "0987",
      "auth_code": "123456",
      "aid": "",
      "mid": "",
      "tid": "",
      "amount": 1000,
      "currency": "GBP"
    },
    {
      "type": "cash",
      "amount": 1000,
```

```
      "currency": "GBP"
    },
    {
      "type": "gift_card",
      "gift_card_type": "One4all",
      "amount": 1000,
      "currency": "GBP"
    }
]
```


Payments tell us how you paid for your purchase. While it will always include a card payment, sometimes a cash payment or a gift card is included as well. All of the payments together should add up to the Receipt total.

The payment details might not appear in the app just yet.

Property	Description
`type` Required	`card`, `cash`, or `gift_card`
`amount` Required	Amount paid in pennies
`currency` Required	e.g. GBP
`last_four`	The last four digits of the card number, for `card` Payments.
`gift_card_type`	A description of the gift card, for `gift_card` Payments

Receipt Merchant

The merchant gives us more information about where the purchase was made, to help us decide what to show at the top of the receipt.

Property	Description
`name`	The merchant name
`online`	`true` for Ecommerce merchants like Amazon `false` for offline merchants like Pret or Starbucks
`phone`	The phone number of the store
`email`	The merchant's email address
`store_name`	The name of that particular store, e.g. `Old Street`
`store_address`	The store's address
`store_postcode`	The store's postcode

Create receipt

```
$ http PUT "https://api.monzo.com/transaction-receipts" \
    "Authorization: Bearer $access_token" \
    # ... JSON Receipt data ...
```


```
{
  "transaction_id": "tx_00...",
  "external_id": "test-receipt-1",
  "total": 1299,
  "currency": "GBP",
  "items": [
    {
      "description": "Bananas, 70p per kg",
      "quantity": 18.56,
      "unit": "kg",
      "amount": 70,
      "currency": "GBP"
    }
  ]
}
```



```
{
    "receipt_id": "receipt_00009NrKwNtI3gKqte",
    ...
}
```


To attach a receipt to a transaction, make a `PUT` request to the `/transaction-receipts` API. Your request should include a body containing the receipt encoded as JSON.

> Unlike our earlier APIs, this API takes a JSON-encoded request body, rather than a form-encoded body.

If you're successful, you'll get back a `200 OK` HTTP response with an empty body. After that, the receipt will show up in your Monzo app!

The `external_id` is used as an idempotency key, so if you call this endpoint again with the same external ID, it will **update** the existing receipt.

Retrieve receipt

You can read back a receipt that you've created based on its external ID.

Note that you'll only be able to read your own receipts in this way.

```
$ http GET "https://api.monzo.com/transaction-receipts" \
    "Authorization: Bearer $access_token" \
    "external_id==test-receipt-1"
```

```
{
  "receipt": {
    "id": "receipt_00009eNJqNeJvKeoQA",
    "external_id": "test-receipt-1",
    ...
  }
}
```

REQUEST ARGUMENTS

Parameter	Description
`external_id` Required	The external ID of the receipt.

Delete receipt

You can delete a receipt based on its external ID.

Note that you can also **update** an existing receipt, by creating it again with different values.

```
$ http DELETE "https://api.monzo.com/transaction-receipts" \
    "Authorization: Bearer $access_token" \
    "external_id==test-receipt-1"
```

```
{}
```

REQUEST ARGUMENTS

Parameter	Description
`external_id` Required	The external ID of the receipt.

Webhooks

Webhooks allow your application to receive real-time, push notification of events in an account.

Registering a webhook

```
$ http --form POST "https://api.monzo.com/webhooks" \
    "Authorization: Bearer $access_token" \
    "account_id=$account_id" \
    "url=$url"
```

```
{
    "webhook": {
        "account_id": "account_id",
        "id": "webhook_id",
        "url": "http://example.com"
    }
}
```

Each time a matching event occurs, we will make a POST call to the URL you provide. If the call fails, we will retry up to a maximum of 5 attempts, with exponential backoff.

REQUEST ARGUMENTS

Parameter	Description
`account_id` Required	The account to receive notifications for.
`url` Required	The URL we will send notifications to.

List webhooks

```
$ http "https://api.monzo.com/webhooks" \
    "Authorization: Bearer $access_token" \
    "account_id=$account_id"
```


```
{
    "webhooks": [
        {
            "account_id": "acc_000091yf79yMwNaZHhHGzp",
            "id": "webhook_000091yhhOmrXQaVZ1Irsv",
            "url": "http://example.com/callback"
        },
        {
            "account_id": "acc_000091yf79yMwNaZHhHGzp",
            "id": "webhook_000091yhhzvJSxLYGAceC9",
            "url": "http://example2.com/anothercallback"
        }
    ]
}
```


List the webhooks your application has registered on an account.

REQUEST ARGUMENTS

Parameter	Description
`account_id` Required	The account to list registered webhooks for.

Deleting a webhook

```
$ http DELETE "https://api.monzo.com/webhooks/$webhook_id" \
    "Authorization: Bearer $access_token"
```


```
{}
```


When you delete a webhook, we will no longer send notifications to it.

Transaction created


```
{
    "type": "transaction.created",
    "data": {
        "account_id": "acc_00008gju41AHyfLUzBUk8A",
        "amount": -350,
        "created": "2015-09-04T14:28:40Z",
        "currency": "GBP",
        "description": "Ozone Coffee Roasters",
        "id": "tx_00008zjky19HyFLAzlUk7t",
        "category": "eating_out",
        "is_load": false,
        "settled": "2015-09-05T14:28:40Z",
        "merchant": {
            "address": {
                "address": "98 Southgate Road",
                "city": "London",
                "country": "GB",
                "latitude": 51.54151,
                "longitude": -0.08482400000002599,
                "postcode": "N1 3JD",
                "region": "Greater London"
            },
            "created": "2015-08-22T12:20:18Z",
            "group_id": "grp_00008zIcpbBOaAr7TTP3sv",
            "id": "merch_00008zIcpbAKe8shBxXUtl",
            "logo": "https://pbs.twimg.com/profile_images/527043602623389696/68_SgUWJ.jpeg",
            "emoji": "🍞",
            "name": "The De Beauvoir Deli Co.",
            "category": "eating_out"
        }
    }
}
```


Each time a new transaction is created in a user's account, we will immediately send information about it in a `transaction.created` event.

Errors

The Monzo API uses conventional HTTP response codes to indicate errors, and includes more detailed information on the exact nature of an error in the HTTP response.

HTTP RESPONSE CODES

Response code	Meaning
`200` OK	All is well.
`400` Bad Request	Your request has missing arguments or is malformed.
`401` Unauthorized	Your request is not authenticated.
`403` Forbidden	Your request is authenticated but has insufficient permissions.
`405` Method Not Allowed	You are using an incorrect HTTP verb. Double check whether it should be `POST`/`GET`/`DELETE`/etc.
`404` Page Not Found	The endpoint requested does not exist.
`406` Not Acceptable	Your application does not accept the content format returned according to the `Accept` headers sent in the request.
`429` Too Many Requests	Your application is exceeding its rate limit. Back off, buddy. :p
`500` Internal Server Error	Something is wrong on our end. Whoopsie.
`504` Gateway Timeout	Something has timed out on our end. Whoopsie.

AUTHENTICATION ERRORS

Errors pertaining to authentication are standard errors but also contain extra information to follow the OAuth specification. Specifically, they contain the `error` key with the following values:

`ERROR` ARGUMENT VALUES

Value	Meaning
`invalid_token`	The supplied access token is invalid or has expired.